

1. Overview Frontend

Il frontend del progetto IoDono è sviluppato come un'applicazione reattiva e modulare, costruita utilizzando la libreria **React** supportata dal build tool **Vite** per garantire prestazioni elevate in fase di sviluppo e produzione.

2 Scelte tecnologiche

La selezione dello stack tecnologico è stata guidata dalla necessità di creare un'applicazione moderna, mantenibile e scalabile, riducendo al contempo il tempo di sviluppo grazie all'uso di librerie esistenti.

2.1 React

- **Scelta:** React è stato scelto come framework fondamentale per la costruzione dell'interfaccia utente.
- **Motivazioni e Vantaggi:**
 - **Component-Based Architecture:** Permette la creazione di componenti riutilizzabili e isolati (presenti nella cartella `components`), facilitando la manutenzione e la scalabilità del codice.
 - **Librerie:** Offre una vasta gamma di librerie di terze parti già pronte, accelerando lo sviluppo di funzionalità complesse.

2.2 Material UI

- **Scelta:** Libreria principale per i componenti grafici.
- **Motivazioni e Vantaggi:**
 - **Velocità di Sviluppo:** Fornisce i componenti essenziali “già pronti” (bottoni, campi input, modali, layout a griglia), riducendo drasticamente il tempo necessario per costruire interfacce gradevoli e funzionali.
 - **Personalizzazione:** I componenti offrono props specifici (es. `sx`) che permettono una personalizzazione puntuale senza dover riscrivere il CSS da zero.

2.3 Tailwind CSS

- **Scelta:** Utilizzato in combinazione con MUI per la gestione del layout e dello styling rapido.
- **Motivazioni e Vantaggi:**

- **Struttura Semplificata:** Elimina la necessità di gestire file `.css` separati per ogni pagina o componente. Tutto lo stile è applicato direttamente tramite classi nel file `.jsx`.
- **Efficienza:** Permette di applicare stili complessi con poche classi concise, riducendo la verbosità del codice rispetto al CSS tradizionale.

2.4 Server Frontend e Build Tool: Vite

- **Scelta:** Utilizzato come server principale per lo sviluppo frontend, strumento di build e gestore delle configurazioni ambientali.
- **Motivazioni e Vantaggi:**
 - **Server Frontend:** Agisce da web server locale durante la fase di sviluppo.
 - **Aggiornamento in tempo reale:** Serve l'applicazione al browser in tempo reale e gestisce il ricaricamento automatico dei moduli, permettendo di visualizzare istantaneamente le modifiche al codice senza perdere lo stato della pagina.
 - **Gestione Variabili d'Ambiente:** Gestisce centralmente le configurazioni che variano tra sviluppo e produzione (come l'URL dell'API).

2.5 Axios

- **Scelta:** Libreria per effettuare richieste HTTP al backend
- **Motivazioni e Vantaggi:**
 - **Interceptors:** Questa è la funzionalità chiave per l'architettura del progetto. Axios permette di definire intercettori che agiscono globalmente su ogni richiesta.
 - **Configurazione Istanza:** Permette di creare un'istanza (`api`) con una `baseURL` preconfigurata e timeout automatici, semplificando la gestione degli endpoint.

3. Architettura

Il progetto adotta un'architettura basata su componenti funzionali, con una netta separazione tra la logica di visualizzazione (UI Components), la gestione dello stato globale (Context API) e la comunicazione dati (Services/Hooks).

3.1 Entry Point (`App.jsx`)

Il file `App.jsx` rappresenta il punto di ingresso logico dell'applicazione. La sua responsabilità principale è l'inizializzazione dei **Global Providers**, che iniettano funzionalità diverse nell'albero dei componenti.

La struttura a “wrapper” include:

- **LocalizationProvider:** Configura la gestione delle date tramite la libreria `Day.js`, impostando la localizzazione italiana (`it`) per i componenti di selezione data (es. `DateTimePicker`).
- **GoogleOAuthProvider:** Inizializza il contesto per l'autenticazione tramite Google Sign-In utilizzando il `CLIENT_ID`.
- **ThemeProvider:** Distribuisce il tema personalizzato di Material UI (definito in `src/utils/materialUITheme.js`) a tutti i componenti.
- **AuthProvider:** Avvia il contesto di autenticazione per gestire la sessione utente.
- **RouterProvider:** Gestisce il routing dell'applicazione passando la configurazione definita in `router.jsx`.

3.2 Overview struttura cartelle

Facendo riferimento all'organizzazione del progetto, ogni cartella ha una responsabilità specifica:

- **assets/:** Contiene risorse statiche (immagini, icone).
- **components/:** Contiene i componenti che costituiscono l'interfaccia, suddivisi in:
 - **form/:** Componenti funzionali per la raccolta dati (input, form di validazione).
 - **ui/:** Componenti puramente visivi per il data display (alert, header, dialog di visualizzazione).
- **context/:** Contiene la gestione dello stato globale (`AuthProvider`, `DonationProvider`).
- **hooks/:** Contiene custom hooks (es. `useAuth`, `useDonation`) per astrarre la logica dai componenti visivi.
- **pages/:** Contiene le viste principali (pagine) che compongono le rotte.
 - Include la cartella `Dashboard/` per le viste specifiche dei ruoli.
- **services/:** Contiene i file per le chiamate agli endpoint del backend (es. `donationService.js`, `loginService.js`), utilizzando l'istanza configurata in `api.js`.
- **utils/:** Contiene funzioni di utilità, costanti, validatori e la configurazione del tema grafico.

3.3 Routing (`router.jsx`)

La navigazione è gestita tramite `react-router-dom` utilizzando `createBrowserRouter`. Il router distingue nettamente tra rotte pubbliche e protette.

- **Rotte Pubbliche:** Accessibili a chiunque (es. `/login`, `/registration`, `/example`).
- **Rotte Protette:** Raggruppate sotto un layout wrapper gestito dal componente `RequireAuth`. Tutte le rotte figlie (es. `/`, `/prot`) richiedono un token valido.

3.3.1 Protezione delle Rotte

Il componente `RequireAuth` agisce come guardia di sicurezza:

1. Recupera il token JWT dal `localStorage`.
2. Utilizza `jwt-decode` per verificare se il token è scaduto confrontando il campo `exp` con l'orario attuale.
3. Se il token è valido e l'utente è presente nel contesto, renderizza il contenuto richiesto (`<Outlet />`).
4. In caso contrario, esegue il logout forzato e reindirizza l'utente alla pagina di login (`<Navigate to="/login" />`).

3.5 Gestione API (`services/api.js`)

Tutte le comunicazioni HTTP verso il backend sono centralizzate tramite un'istanza personalizzata di `Axios`.

- **Configurazione Dinamica:** L'URL di base (`baseUrl`) viene determinato a runtime: in ambiente di sviluppo (DEV) utilizza la variabile d'ambiente `VITE_BACKEND_API` (o proxy), mentre in produzione punta al percorso relativo `/api`.
- **Request Interceptor:** È implementato un intercettore che inietta automaticamente il token di autenticazione in ogni richiesta. Prima dell'invio, il sistema preleva l'`authToken` dal `localStorage` e lo aggiunge all'header `Authorization` nel formato `Bearer {token}`. Questo elimina la necessità di gestire l'autenticazione manualmente nelle singole chiamate ai servizi.

3.6 Componente `Home.jsx`

La pagina `Home` funge da **Switch Logico** basato sul ruolo dell'utente

Invece di avere una singola dashboard, il componente esamina `user.role` (fornito dall'`AuthProvider`) e renderizza condizionalmente il componente appropriato:

- Se `role === ASSOCIATION` → Mostra `<AssociationDashboard />`.

- Se `role === DONOR` → Mostra `<DonorDashboard />`.
- Se `role === ADMIN` → Mostra `<AdminDashboard />`.

Questo garantisce che ogni tipologia di utente visualizzi esclusivamente le funzionalità pertinenti al proprio ruolo.

3.7 Gestione dello Stato Globale (Context API)

Per gestire dati condivisi, sono stati implementati due Context principali.

3.7.1 AuthProvider

Responsabile della gestione dell'identità dell'utente.

- **Persistenza:** Al caricamento (`useEffect`), controlla `localStorage` per recuperare eventuali sessioni attive (`authToken` e oggetto `user`), ripristinando lo stato senza richiedere un nuovo login.
- **Login/Logout:** Espone i metodi `login` (che salva token e dati utente nello storage) e `logout` (che pulisce lo storage e resetta lo stato).

3.7.2 DonationProvider

Responsabile della sincronizzazione dei dati delle donazioni tra backend e frontend.

- **Gestione Stato Multiplo:** Mantiene tre stati separati per le donazioni: `activeDonations`, `acceptedDonations` e `completedDonations`.
- **Dati Derivati:** Utilizza `useMemo` per calcolare `allDonations`, un array aggregato di tutte le donazioni, ricalcolato solo quando i dati sottostanti cambiano.
- **Ottimizzazione Fetch:** La funzione `fetchDonations` utilizza `Promise.all` per eseguire le chiamate API in parallelo, riducendo i tempi di caricamento.
- **Modalità Debug:** Supporta una `DEBUG_MODE` (tramite variabili d'ambiente) che, se attiva, bypassa il backend e serve dati statici di esempio (`activeEx`, etc.) per facilitare lo sviluppo offline.
- **UI Optimistic:** Fornisce la funzione `removeDonationLocally` per aggiornare immediatamente l'interfaccia utente (rimuovendo una donazione dalle liste) prima o durante la conferma dal backend.

4. Flusso di Autenticazione Frontend

Il sistema supporta due modalità distinte di accesso e registrazione. Sebbene entrambe condividono la generazione di un token JWT per la gestione della

sessione, i passaggi iniziali differiscono significativamente per gestire il ruolo dell'utente nella maniera corretta

4.1 Email/Password

In questo scenario, l'utente è responsabile dell'inserimento di tutti i dati necessari.

1. Registrazione Completa:

- L'utente accede alla pagina `/registration`.
- Compila manualmente tutti i campi del form: Nome, Email, Password, Telefono, Indirizzo e seleziona il **Ruolo** (Donatore o Associazione).
- Al submit, i dati vengono inviati al backend che crea l'utente nel database.

2. Login Esplicito:

- Dopo la registrazione, l'utente viene reindirizzato al login.
- Inserisce Email e Password.
- Il backend valida le credenziali e restituisce il token JWT e l'oggetto User.
- il frontend salva il token JWT e l'oggetto user nel local storage

4.2 Google OAuth

Questo flusso delega l'identificazione a Google ma richiede un passaggio intermedio per raccogliere i dati specifici della piattaforma (Ruolo e Telefono) che non sono presenti nel profilo Google.

1. Iniziazione (Login o Registrazione):

- L'utente clicca sul pulsante "Accedi con Google" (gestito da `GoogleOAuthProvider`).
- Si apre il pop-up di Google per il consenso.
- Alla conferma, Google restituisce un token temporaneo al frontend.

2. Verifica Backend:

- Il frontend invia il token di Google al backend.
- Il backend verifica l'identità e controlla se l'email è già presente nel database.

3. Caso A: Utente Già Registrato: Se l'email esiste, il backend considera l'utente autenticato e restituisce immediatamente il token JWT finale (Login completato).

4. **Caso B: Nuovo Utente (Registrazione Incompleta):** Se l'email non esiste, il backend segnala che l'utente è nuovo.
 - Il frontend reindirizza l'utente alla pagina di **Registrazione Semplificata**.
 - **Form Adattivo:** I campi "Email" e "Password" vengono nascosti (poiché gestiti da Google).
 - L'utente deve obbligatoriamente selezionare il **Ruolo**
 - Al submit, i dati integrativi vengono uniti al profilo Google per creare l'utente definitivo.

4.3 Fase Comune

Indipendentemente dal metodo utilizzato (Standard o Google), una volta ottenuto il JWT finale:

1. **Storage Sicuro:** L'AuthProvider intercetta il token e lo salva nel `localStorage`.
2. **Configurazione API:** Da questo momento, l'interceptor di Axios (`api.js`) inietterà automaticamente il token nell'header `Authorization` di ogni richiesta.
3. **Routing:** Il componente `RequireAuth` valida il token e permette l'accesso alle rotte protette, mentre `Home.jsx` smista l'utente alla dashboard corretta.